

Salesforce/SFR-Embedding-2_R
 like 95

Follow  Salesforce AI Research 2.44k

 Feature Extraction
 sentence-transformers
 Safetensors
 Transformers
 English

mistral
mteb
 Eval Results (legacy)
 text-embeddings-inference
 License: cc-by-nc-4.0

 Deploy

 Copy to bucket **NEW**

Use this model


Model card

Files

 xet


Community 8

Salesforce/SFR-Embedding-2_R

SFR-Embedding by Salesforce Research.

The model is for **research purposes only**.

More technical details will be updated later.

Meanwhile, please refer to our previous work [SFR-Embedding](#) for details.

Ethical Considerations

This release is for research purposes only in support of an academic paper. Our models, datasets, and code are not specifically designed or evaluated for all downstream purposes. We strongly recommend users evaluate and address potential concerns related to accuracy, safety, and fairness before deploying this model. We encourage users to consider the common limitations of AI, comply with applicable laws, and leverage best practices when selecting use cases, particularly for high-risk scenarios where errors or misuse could significantly impact people's lives, rights, or safety. For further guidance on use cases, refer to our [AUP](#) and [AI AUP](#).

Downloads last month

17,830


Safetensors

Model size 7B params

Tensor type BF16  Files info

Inference Providers **NEW**

 Feature Extraction

This model isn't deployed by any Inference Provider.

 Ask for provider support

Model tree for Salesforce/SFR-Em...

Finetunes 1 model

Quantizations 2 models

Spaces using Salesforce/SFR-... 27

 mteb/leaderboard

 MTEB-BR/leaderboard

 mteb/arena

SFR-Embedding Team (*indicates equal contributors, † indicates co-leaders).

- Rui Meng*
- Ye Liu*
- Tong Niu
- Shafiq Rayhan Joty
- Caiming Xiong †
- Yingbo Zhou †
- Semih Yavuz †

Citation

```
@misc{SFR-embedding-2,  
  title={SFR-Embedding-2: Advanced Text Embedding},  
  author={Rui Meng*, Ye Liu*, Shafiq Rayhan Joty, Caiming Xiong†, Yingbo Zhou†, Semih Yavuz†},  
  year={2024},  
  url={https://huggingface.co/Salesforce/SFR-Embedding-2},  
}
```

How to run

Transformers

The models can be used as follows:

```
import torch  
import torch.nn.functional as F  
from torch import Tensor  
from transformers import AutoTokenizer, AutoModel  
  
def last_token_pool(last_hidden_states: Tensor,  
                    attention_mask: Tensor) -> Tensor:  
    left_padding = (attention_mask[:, -1].sum(dim=1) != 0).bool()  
    if left_padding:  
        return last_hidden_states[:, -1]
```

 mteb/leaderboard_legacy

 Nymbo/MTEB-Arena

 sq66/leaderboard_legacy

 reader-1/1

 shiwan7788/leaderboard-uni

+ 19 Spaces

Collection including Salesforce/S...

SFR-Embedding Models  Collection

A collectio... • 4 items • Updat... •  3

Evaluation results

accuracy ...	test set	self-reported	92.716
ap on MTE...	test set	self-reported	69.479
f1 on MTE...	test set	self-reported	88.025
accuracy ...	test set	self-reported	97.311
ap on MTE...	test set	self-reported	96.267
f1 on MTE...	test set	self-reported	97.310
accuracy ...	test set	self-reported	61.040
f1 on MTE...	test set	self-reported	60.786

▼ Expand 697 evaluations

```

else:
    sequence_lengths = attention_mask.sum(dim=-1)
    batch_size = last_hidden_states.shape[0]
    return last_hidden_states[torch.arange(batch_size, device=device)]

def get_detailed_instruct(task_description: str) -> str:
    return f'Instruct: {task_description}\nQuery:'

# Each query must come with a one-sentence instruction
task = 'Given a web search query, retrieve relevant passages'
queries = [
    get_detailed_instruct(task, 'How to bake a chocolate cake'),
    get_detailed_instruct(task, 'Symptoms of the flu'),
]

# No need to add instruction for retrieval documents
passages = [
    "To bake a delicious chocolate cake, you need to mix flour, sugar, and eggs.",
    "The flu, or influenza, is an illness caused by a virus.",
]

# load model and tokenizer
tokenizer = AutoTokenizer.from_pretrained('Salesforce/sbert-bert')
model = AutoModel.from_pretrained('Salesforce/sbert-bert')

# get the embeddings
max_length = 4096
input_texts = queries + passages
batch_dict = tokenizer(input_texts, max_length=max_length)
outputs = model(**batch_dict)
embeddings = last_token_pool(outputs.last_hidden_state, batch_first=True)

# normalize embeddings
embeddings = F.normalize(embeddings, p=2, dim=-1)
scores = (embeddings[:2] @ embeddings[2:].T)
print(scores.tolist())
# [[40.132083892822266, 25.032529830932617],

```

Sentence Transformers

```

from sentence_transformers import SentenceTransformer

model = SentenceTransformer("Salesforce/SFR-1")

def get_detailed_instruct(task_description: str) -> str:
    return f'Instruct: {task_description}\nQuery:'

# Each query must come with a one-sentence instruction
task = 'Given a web search query, retrieve relevant passages'
queries = [
    get_detailed_instruct(task, 'How to bake a chocolate cake'),
    get_detailed_instruct(task, 'Symptoms of the flu')
]

# No need to add instruction for retrieval documents
passages = [
    "To bake a delicious chocolate cake, you need to mix flour, sugar, and eggs.",
    "The flu, or influenza, is an illness caused by a virus."
]

embeddings = model.encode(queries + passages)
scores = model.similarity(embeddings[:2], embeddings[2:])
print(scores.tolist())
# [[40.13203811645508, 25.032546997070312],

```

